# 1. Orientation & Thesis: Closure of Compositional Hierarchies in Digital Symmetries

This opening section fixes the vocabulary and scope for the rest of the book. The central claim is that many digital systems admit a common compositional description: once the relevant objects, symmetries, and composition laws are identified, one can ask when a class of constructions is *closed* under those laws and how such closure induces a *hierarchy* of increasingly expressive or increasingly constrained models.

We begin with the basic objects of study. In the Boolean setting, the ambient space is $\mathbb{B}^n$, the set of length-$n$ bit vectors. From this starting point we will move among several closely related digital objects: Boolean functions $f : \mathbb{B}^n \to \mathbb{B}^m$, circuits, finite automata, and error-correcting codes. These are not treated as unrelated examples; rather, they are instances of structured finite systems whose behavior can be compared by composition, transformation, and equivalence. The book will repeatedly translate between a semantic view, where one studies the input–output behavior of an object, and a structural view, where one studies the wiring, state, or generating data that realizes it.

To compare such objects, we need a notion of error or distance that respects the level of description. At the level of a Boolean function, a natural metric is the truth-table Hamming distance, denoted $\varepsilon(r)$ when the object is viewed as a function of arity $r$. This measures disagreement between truth tables and is well suited to function-level approximation and classification. At the level of a circuit, the relevant notion is structural edit distance on the circuit graph: how many local changes are required to transform one wiring diagram into another. The two metrics serve different purposes, and the distinction matters throughout the book. Function-level proximity does not necessarily imply structural proximity, and vice versa. In later chapters, this separation will support multi-resolution comparisons between semantic equivalence, approximate equivalence, and implementation-level similarity.

Symmetry is the second organizing principle. A digital object may be invariant under a group action, or it may transform equivariantly. In the Boolean setting, the most prominent symmetries arise from permutations of inputs, encoded by the symmetric group $\Sigma_n$, together with input and output negations. These generate the familiar NPN equivalence relation on Boolean functions: two functions are equivalent if one can be obtained from the other by permuting inputs, negating selected inputs, and optionally negating the output. Symmetry analysis will later be used both as a classification tool and as a design constraint. Invariance identifies features that remain unchanged under a symmetry action, while equivariance records the compatible transformation law of an output under transformed inputs.

Composition is the third pillar. The book uses several composition operations, each appropriate to a different level of abstraction. Sequential composition is written $\circ$, parallel or tensor composition is written $\otimes$, and feedback or trace is written Tr. These operations are not merely notation: they are expected to satisfy structural laws such as associativity, symmetry, and functoriality, so that complex systems can be assembled from smaller ones in a controlled way. The guiding idea is that a useful formalism should make composition explicit and should preserve the relevant equivalences under composition. This is the sense in which the book treats digital systems as compositional objects rather than as isolated artifacts.

The thesis of the book can now be stated informally. Digital systems often organize into compositional hierarchies: small components are combined into larger modules, modules into subsystems, and subsystems into architectures. A hierarchy is meaningful only when the composition laws are stable enough to support abstraction, and closure properties determine which operations remain

inside a chosen class. In this sense, closure is not merely a set-theoretic property; it is a design principle that governs whether a family of digital objects can be manipulated internally without leaving the formalism. Hierarchy, correspondingly, is not just a ranking by size or complexity, but an organized stratification by expressive power, structural constraints, and closure strength.

We will use the term *closure* for the property that a class of objects is preserved under specified operations such as composition, tensoring, feedback, symmetry actions, or other admissible transformations. We will use the term *hierarchy* for an organized stratification of classes by expressive power, structural complexity, or closure strength. A hierarchy may be defined by increasing arity, by richer generators, by additional symmetries, or by stronger notions of equivalence. The point of the book is not that there is a single canonical hierarchy, but that many useful hierarchies can be studied with the same compositional language. This perspective will later connect Boolean clones, operadic generation, circuit families, and symmetry classes under a common framework.

A useful way to read the rest of the book is to keep three questions in view. First: what are the objects, and at what level are they being compared—semantic behavior, structural realization, or both? Second: what symmetries are being quotiented out, preserved, or exploited? Third: which compositions are allowed, and what closure properties do they induce? These questions recur in the foundational chapters on operads and PROPs, in the closure theory of Boolean functions, and in the later applications to circuits, automata, codes, and learning systems.

This orientation section therefore serves as a map. It introduces the objects $\mathbb{B}^n$, circuits, finite automata, and codes; the metrics $\varepsilon(r)$ and structural edit distance; the symmetry notions of invariance, equivariance, and NPN classes; and the composition laws $\circ$, $\otimes$, and Tr. Later chapters will make these ideas precise in the language of operads, PROPs, closure operators, and clone theory, but the conceptual target remains the same: to understand when digital composition is closed, how symmetry constrains that closure, and how hierarchies emerge from the interaction of both.

In the chapters that follow, these terms will be sharpened in progressively more formal settings. The foundational material will introduce the categorical and algebraic machinery needed to state closure theorems precisely, while the later parts of the book will show how the same vocabulary applies across Boolean functions, circuits, automata, codes, and symmetry-aware learning systems. The present section is intentionally lightweight: it does not prove the main theorems, but it fixes the interpretation of the main nouns and verbs of the book so that the subsequent development can proceed without ambiguity.